

Information Retrieval 7

Neural Ranking Models: Long Document Ranking

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Transformer = Architecture that captures context across the entire sequence simultaneously via Self-Attention

BERT = Transformer Encoder + self-supervised learning (MLM/NSP). Resolves polysemy with contextualized embeddings

Fine-tuning = Specializes a pre-trained model for ranking tasks. Pointwise CE loss is the standard

Training techniques = Hard Negative Mining, TCP, and multi-step FT significantly affect performance

In this lecture, we will learn the most basic model for applying BERT to retrieval, **monoBERT**, and methods for handling long documents: **Birch** and **BERT-MaxP**

Understand in detail the simplest way to use BERT for ranking (monoBERT) and learn methods to rank long documents beyond the 512-token input length limitation

First Half: monoBERT

- Design and input format of monoBERT

- Performance on MS MARCO and ablation experiments

- Why BERT is effective for retrieval

- Computational cost of Cross-encoders and Retrieve-and-Rerank

Second Half: Long Document Ranking

- The 512-token wall and the long document problem

- Birch: Sentence-level splitting and score aggregation

- BERT-MaxP: Passage score aggregation

- CEDR, PARADE, and the design space

- Efficiency-accuracy tradeoff

monoBERT

Simple Relevance Classification

monoBERT [Nogueira and Cho, 2019] = The **first** model to apply BERT to document ranking

Input format (Cross-encoder):

$[[CLS], q, [SEP], d_i, [SEP]]$

Concatenates query q and candidate document d_i as input to BERT

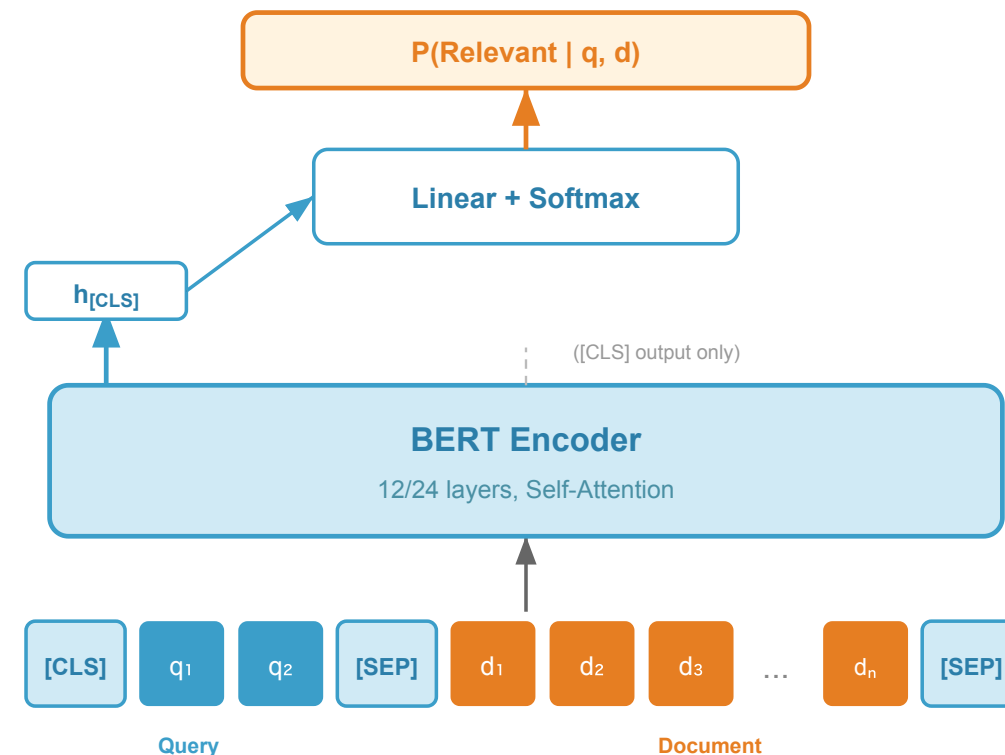
Computes the relevance score from the output representation of the $[CLS]$ token

Scoring:

$$P(\text{Relevant} = 1 \mid d_i, q) = \text{softmax}(T_{[CLS]} W + b)_1$$

$T_{[CLS]} \in \mathbb{R}^D$: Contextualized embedding of $[CLS]$

$W \in \mathbb{R}^{D \times 2}, b \in \mathbb{R}^2$: Classification layer parameters



All tokens from query and document compute mutual Attention, resulting in high accuracy but also high computational cost → **Cross-encoder**

Training data: MS MARCO passage ranking

~530K queries, each with one relevant passage

Negatives: Sampled non-relevant passages from BM25 top results

Loss function: Pointwise Cross-Entropy

$$\mathcal{L} = - \sum_i [y_i \log P(1 \mid d_i, q) + (1 - y_i) \log P(0 \mid d_i, q)]$$

$y_i \in \{0, 1\}$: Relevant/non-relevant label

Each query is trained with one relevant and one non-relevant example as a pair

Hyperparameters:

Learning rate: 3×10^{-6} (lower than the BERT standard)

Batch size: 32

Epochs: ~400K iterations

Training Pipeline

MS MARCO: (q, d⁺, d⁻) triplets

▼ Convert to input format

[CLS] q [SEP] d⁺ [SEP] → y=1
[CLS] q [SEP] d⁻ [SEP] → y=0

▼ BERT + classification layer

Cross-Entropy Loss

▼ Update all parameters

Fine-tuned monoBERT

All layers of the pre-trained BERT are fine-tuned. The classification layer is newly added (randomly initialized)

Results on MS MARCO passage ranking:

Method	MRR@10 (Dev)	MRR@10 (Test)
IRNet (Best Pre-BERT)	0.278	0.281
BM25 (Microsoft impl.)	0.167	0.165
BM25 (Anserini)	0.187	0.190
+ monoBERT (Large)	0.372	0.365
+ monoBERT (Base)	0.347	—

BM25 + monoBERT improves MRR@10 by **roughly 2x**

BERT-Large outperforms BERT-Base

Since this is reranking, **Recall does not change**
(documents missed by BM25 cannot be recovered)

This result was released in January 2019 and became a turning point symbolizing the rapid expansion of BERT-based retrieval research. A relatively simple method significantly outperformed the leading baselines at the time.

TREC DL 2019 — monoBERT on deep relevance judgments:

Method	nDCG@10	MAP
BM25 (Anserini)	0.506	0.301
BM25 + monoBERT (Large)	0.738	0.506

Reranking BM25 with monoBERT lifts nDCG@10 from 0.506 to **0.738 (+0.23, ~46%)**

The **MAP** gain is even larger (0.301 → 0.506);
Recall@1k stays 0.750 — monoBERT only reranks the BM25 candidates

Deep, NIST-assessed graded judgments reveal gains hidden by MS MARCO's sparse judgments

Differences Between MS MARCO and TREC DL

MS MARCO:

Evaluation: Automatic (MRR@10)

Judgments: Shallow (1 relevant passage per query)

Query count: ~530K (Dev: 6,980)

TREC DL 2019:

Evaluation: Manual by NIST assessors

Judgments: Deep (4-level graded relevance)

Query count: 43 (deep judgments)

Improvements invisible in MS MARCO's shallow judgments were confirmed by TREC DL's deep judgments

Comparison with Pre-BERT models:

Representation-based (DSSM): Encodes q and d **independently**

Interaction-based (DRMM): Captures interactions via **similarity matrices**

monoBERT: All-to-all Attention captures **all interactions** simultaneously

Information BERT captures:

1. **Exact match**: Query terms appearing verbatim in the document
2. **Semantic match**: Synonym and paraphrase matching
3. **Word position information**: Word positions and syntactic roles

Concrete Example

Query: "international art crime"

Most important sentence in the relevant document:

"Three armed **robbers** take 21 Renaissance **paintings** worth more than \$5 million from a gallery in Zurich, Switzerland."

→ Not a single query term appears!

Attention analysis:

"art"



"paintings" (semantic match)

"crime"



"robbers" (semantic match)

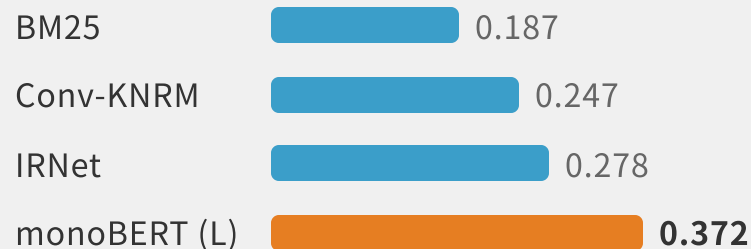
Comparison with Pre-BERT models on MS MARCO passage (Dev):

Model	Type	MRR@10
BM25 (Anserini)	Lexical	0.187
KNRM	Interaction	0.218
Conv-KNRM	Interaction	0.247
IRNet	Representation + Interaction	0.278
monoBERT (Base)	Cross-encoder	0.347
monoBERT (Large)	Cross-encoder	0.372

monoBERT exceeds the best Pre-BERT model (IRNet) by **+0.094**

The gap over interaction-based models (KNRM family) is **+0.125** or more

MRR@10 Comparison (Conceptual diagram)



Conceptual diagram showing general trends

BERT's all-to-all Attention significantly outperforms the partial interaction capture of previous approaches

Cross-encoder (scope of this lecture):

$$\text{score}(q, d) = f_{\theta}([\text{CLS}] \ q \ [\text{SEP}] \ d \ [\text{SEP}])$$

Concatenates q and d as input

Mutual Attention across all tokens

Accuracy: High (captures all interactions)

Speed: Slow (requires inference per document)

Use case: Reranking (Stage 2)

Bi-encoder (covered in a future lecture):

$$\text{score}(q, d) = \text{sim}(f_{\theta}(q), g_{\phi}(d))$$

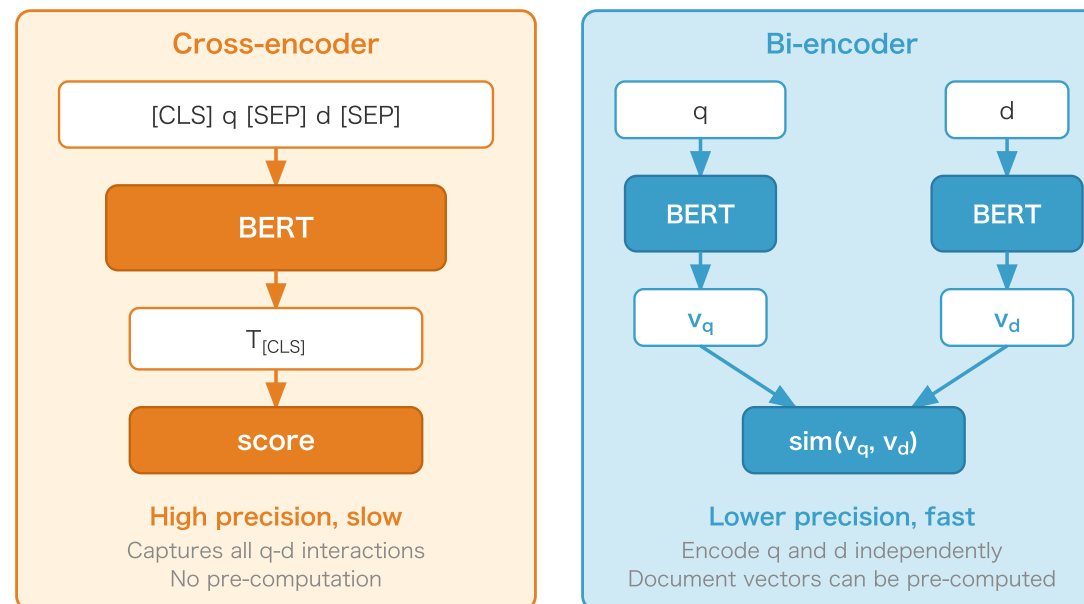
Encodes q and d **independently**

Document representations can be **pre-computed**
and cached

Accuracy: Lower than Cross-encoder

Speed: Fast (approximate nearest neighbor search available)

Cross-encoder vs Bi-encoder



Computational complexity of Self-Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Computing $QK^\top$: $O(n^2 \cdot d)$ — Proportional to the **square** of input length n

BERT-Base ($n = 512$, 12 layers): ~**32M** attention computations/layer

Memory usage is also $O(n^2)$: Attention matrices must be stored for all layers

Inference latency estimates (GPU, BERT-Base):

Input length	Latency (per item)
128 tokens	~5 ms
256 tokens	~12 ms
512 tokens	~30 ms

Why "Reranking"?

Cost estimate for full corpus scoring:

Corpus: 8.8M passages (MS MARCO)

BERT inference: ~30ms per item

Full corpus scoring:

8.8M x 30ms = **73 hours/query**

→ Full corpus Cross-encoder scoring is **practically impossible**

The Cross-encoder dilemma: Accuracy is high, but since query and document must be concatenated as input, independent inference is required per document. Pre-computation and caching are impossible

A **two-stage architecture** that balances computational cost and accuracy:

Stage 1: Candidate Retrieval

BM25 + inverted index

Retrieves 1,000 candidates in **~10ms**

High Recall, moderate Precision

Stage 2: Reranking

monoBERT (Cross-encoder)

Re-scores the top k (e.g., $k = 100$) candidates

100 items x 30ms = **~3 seconds**

Specific latency estimates:

k (reranking depth)	Latency
10	~0.3s
100	~3s
1,000	~30s

Retrieve-and-Rerank Flow

Corpus (8.8M documents)

▼ BM25 (~10ms)

Retrieve top 1,000

Recall@1000 $\approx$ 0.86 (MS MARCO)

▼ monoBERT (~3s, $k=100$)

Rerank top 100

MRR@10: 0.187 $\rightarrow$ 0.372

Final ranking (top 10)

BM25's Recall determines the performance ceiling of reranking.
Documents missed by BM25 cannot be recovered by BERT

Long Document Ranking

From Passage to Document Ranking

Problem: BERT's maximum input length is **512 tokens** (WordPiece)

MS MARCO passage: Average 56 words → Fits within 512

News articles: Average 500-1,000 words → Does not fit

Scientific papers: Thousands of words → Significantly exceeds

Two challenges:

1. **Training:** Relevance labels are assigned at the document level, but BERT can only take passage-level inputs. It is unclear which passages contain relevance information
2. **Inference:** The entire document cannot be fed to BERT. How to split and how to aggregate scores

Aggregation Approaches

Score Aggregation

Compute a score for each chunk, then aggregate via max/sum, etc.
Example: BERT-MaxP

Representation Aggregation

Combine representations from each chunk before computing the score
Example: CEDR, PARADE

Score aggregation is simple to implement and differentiable (end-to-end trainable). Representation aggregation is more flexible but more complex

Three splitting strategies for feeding long documents into BERT:

1. Sentence-level splitting

Split the document at sentence boundaries

Each sentence is short, guaranteed to fit within 512 tokens

Significant context loss (no information from surrounding sentences)

Representative method: **Birch**

2. Passage-level splitting

Split using a fixed-length sliding window

Overlap mitigates context loss

Ignores sentence boundaries, so sentences may be cut mid-way

Representative method: **BERT-MaxP** (150 words, stride 75 words)

Comparison of Splitting Strategies

Sentence level

Chunk count: Many (20-50 sentences per document)

Context retention: **Low** / Implementation: **Easy**

Passage level (Sliding window)

Chunk count: Moderate (5-15 per document)

Context retention: **Medium** / Implementation: **Easy**

Paragraph level

Chunk count: Few (3-10 per document)

Context retention: **High** / Implementation: **Depends on document structure**

In practice, passage-level splitting (sliding window) is the most widely used

Birch [Akkalyoncu Yilmaz et al., 2019] approach:

1. **Training:** Avoids the length problem — Trains BERT on short texts (e.g., tweets) and **transfers the relevance judgment ability via zero-shot**
2. **Inference:** Splits the document into **sentences**, pairing each sentence with the query for BERT inference

Document score computation:

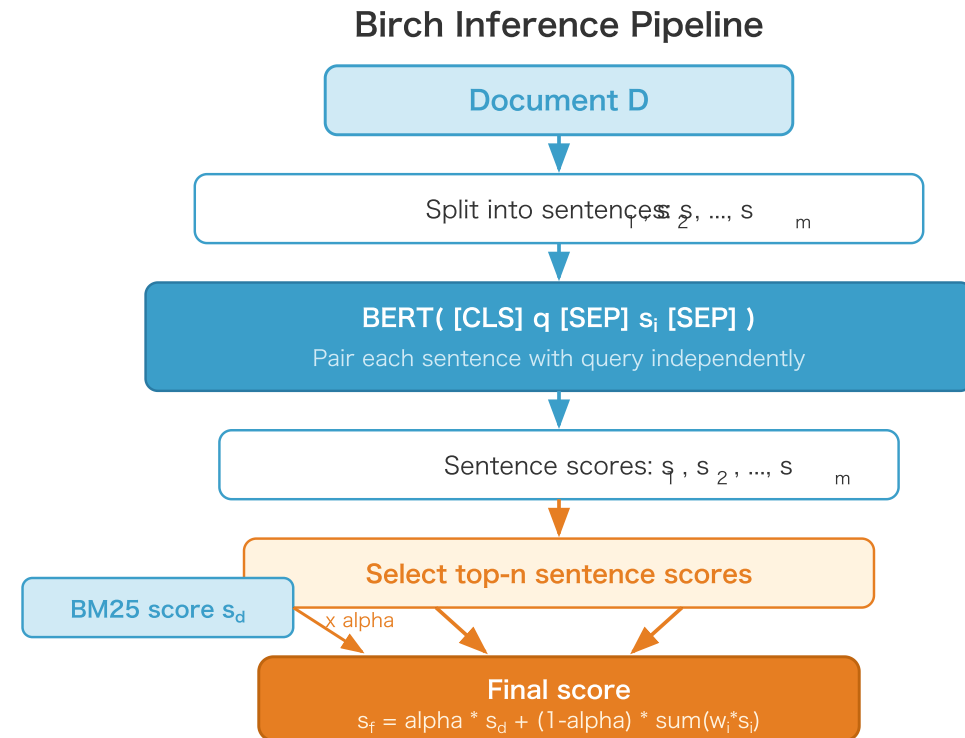
$$s_f = \alpha \cdot s_d + (1 - \alpha) \cdot \sum_{i=1}^n w_i \cdot s_i$$

s_d : BM25 document score

s_i : BERT score for the top n sentences

w_i : Weight for each sentence (tuned via cross-validation)

α : Interpolation parameter between BM25 and BERT



BERT-MaxP [Dai and Callan, 2019] approach:

1. **Training:** Split documents into overlapping passages, treating all passages from relevant documents as relevant and all from non-relevant documents as non-relevant (**noisy labels**)
2. **Inference:** Use the **maximum** BERT score across passages as the document score

Passage splitting:

150-word sliding window, stride 75 words

Each passage: $[[CLS], q, [SEP], p_i, [SEP]]$

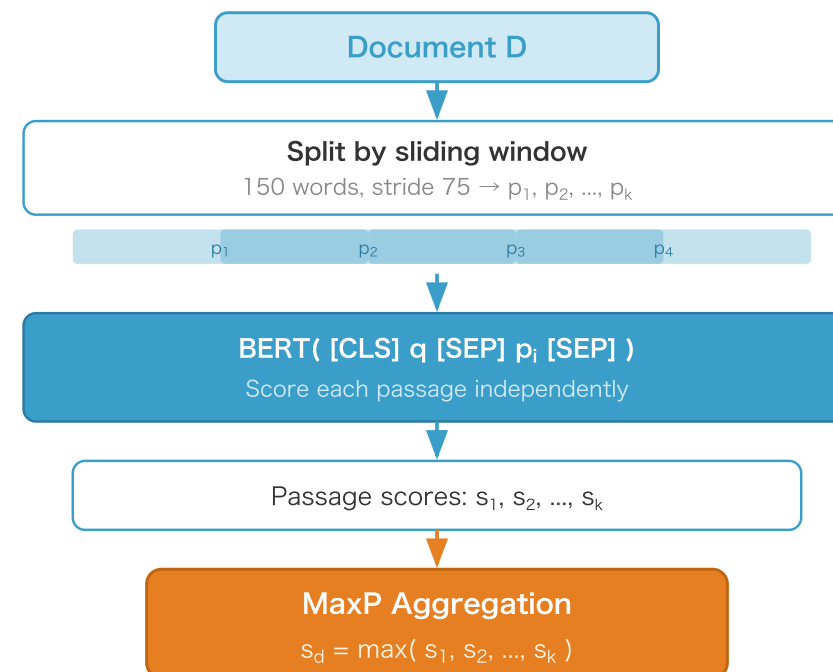
Comparison of aggregation strategies:

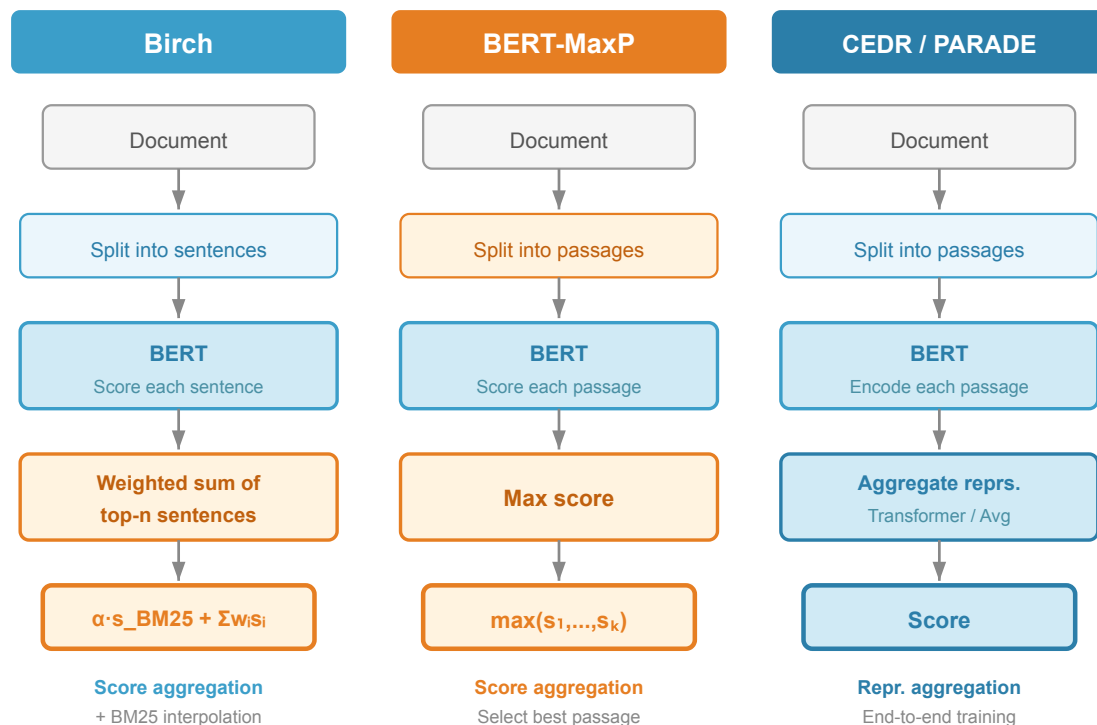
MaxP: $s_d = \max_i s_i$ (best passage)

FirstP: $s_d = s_1$ (first passage)

SumP: $s_d = \sum_i s_i$ (sum of all passages)

BERT-MaxP Inference Pipeline





Performance is **roughly equivalent** (Robust04 nDCG@20: 0.529 vs 0.533)

Design philosophies differ, but the shared insight is that "a document's relevance is determined by its most relevant portion"

Both significantly outperform the best Pre-BERT methods and represent the "first wave" of BERT document ranking models

CEDR/PARADE offer more flexibility through **representation aggregation** but increase implementation and computational costs

Systematic organization of BERT ranking methods for long documents:

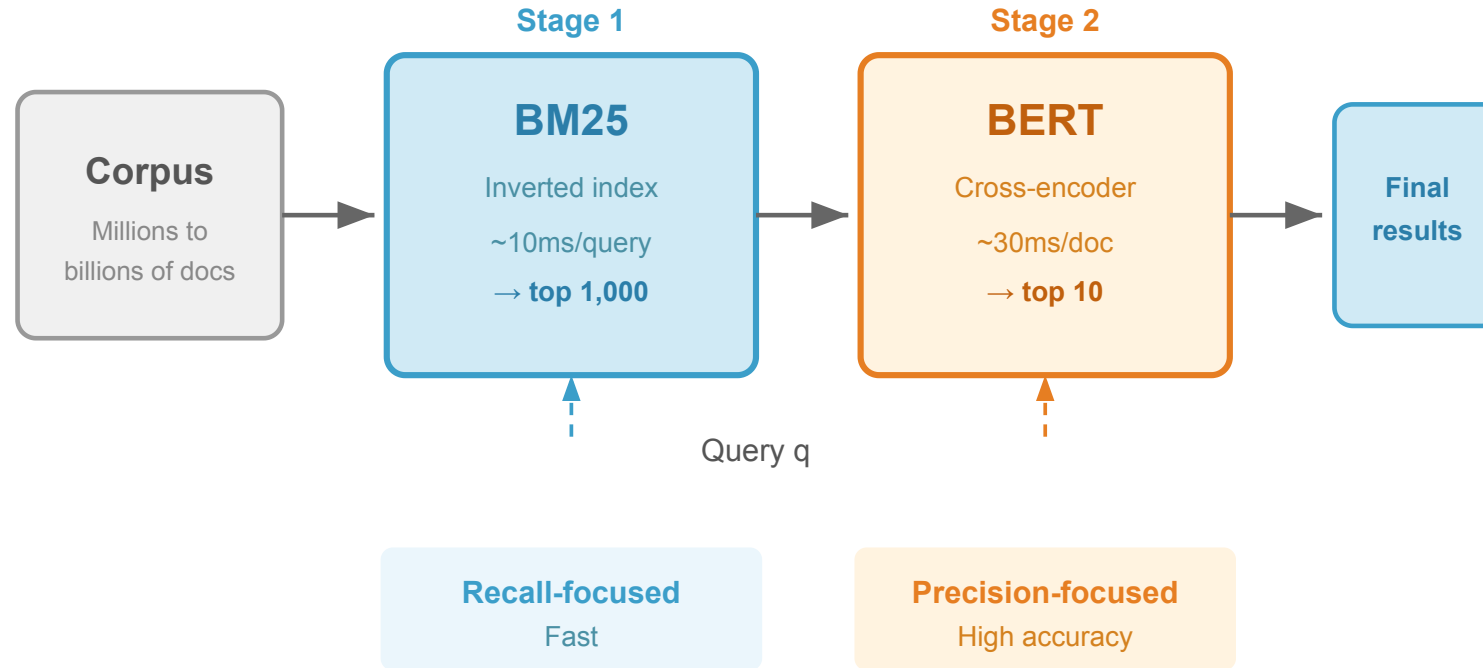
Method	Split unit	Aggregation	Training strategy	BM25 integration
BERT-FirstP	Passage	First only	Passage FT	None
BERT-MaxP	Passage	Score max	Passage FT (noisy labels)	None
BERT-SumP	Passage	Score sum	Passage FT (noisy labels)	None
Birch	Sentence	Weighted score sum	Zero-shot transfer	Interpolation
CEDR	Chunk	Repr. aggregation + ranking model	End-to-end	None
PARADE	Passage	Repr. aggregation (Transformer)	End-to-end	None

Design lessons:

Max aggregation is generally strong → A document's relevance is determined by its most relevant portion

Representation aggregation has a higher performance ceiling but increases implementation and computational costs

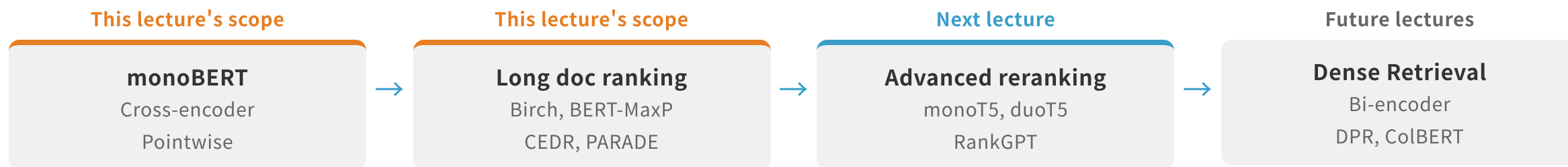
All these methods are **engineering solutions for handling long documents within the 512-token constraint**. Long-sequence Transformers like Longformer and BigBird take a different approach (not covered in this lecture)



This **Retrieve-and-Rerank** architecture is the current standard

Stage 1 (BM25): Fast → **Securing Recall** | Stage 2 (BERT): Accurate → **Improving Precision**

"Coarse retrieval → precise reranking" is the **fundamental principle** for resolving the efficiency-accuracy tradeoff



What we learned today:

- The power and computational cost of Cross-encoders
- Fundamental principles of Retrieve-and-Rerank
- Methods for handling long documents (splitting and aggregation)

Next challenges:

- Pointwise → Pairwise/Listwise reranking
- Cross-encoder → Bi-encoder (using BERT for retrieval too)
- Dense Retrieval to break through BM25's Recall ceiling

Lin et al., "Pretrained Transformers for Text Ranking: BERT and Beyond", 2021

1. **monoBERT** = A simple ranking model using BERT. As a Cross-encoder, it takes [CLS] q [SEP] d [SEP] as input and estimates relevance from the [CLS] representation. Reported to significantly outperform BM25 on MS MARCO
2. **Retrieve-and-Rerank** = Resolves the Cross-encoder's $O(n^2)$ computational cost with a two-stage configuration. BM25 secures Recall while BERT improves Precision
3. **Birch / BERT-MaxP** = Split long documents into sentences/passages and score with BERT. The shared insight is that "a document's relevance is determined by its most relevant portion"
4. **Design space** = Two directions: score aggregation (MaxP) and representation aggregation (CEDR, PARADE). Pointwise-only, slowness, and Recall ceiling remain as open challenges

Next Lecture Preview: Cross-Encoder Reranking

monoT5 (Seq2Seq ranking)

duoT5 (Pairwise reranking)

RankGPT (LLM-based Listwise reranking)